Unified Room Allocation and Lecture Hall Complex Management System: A Web-Based Solution for Academic Resource Scheduling

Aryan Ashok Jain
Requirements, Software System
Indian Institute of Technology Jodhpur
Jodhpur, India
b24cs1016@iitj.ac.in

Anshika Jha
Requirements, Team
Indian Institute of Technology Jodhpur
Jodhpur, India
b24cs1010@iitj.ac.in

Suparn Agrawal
Requirements, Work, Opportunity
Indian Institute of Technology Jodhpur
Jodhpur, India
b24cs1100@iitj.ac.in

Rewant Rai
Requirements, Stakeholders, Way of Working
Indian Institute of Technology Jodhpur
Jodhpur, India
b24cs1091@iitj.ac.in

***Abstract*—The management of lecture halls and classroom resources in academic institutions presents significant operational challenges, particularly when dependent on manual or spreadsheet-based booking mechanisms. This paper presents the conceptual design and requirements engineering process of a Unified Room Allocation and Lecture Hall Complex (LHC) Management System—a web-based platform intended to automate classroom booking, timetable-driven allocation, slot management, and, in future phases, examination seating arrangements.**

The work conducted thus far has focused on iterative refinement of system requirements through seven structured revisions of the Software Requirements Specification (SRS), developed in accordance with IEEE standards. UML-based modeling was employed to progressively formalize functional and non-functional requirements, resolve ambiguities, and ensure traceability across evolving design decisions.

A review of existing room management practices identified limitations in scalability, inclusive student access, and automated conflict resolution. The proposed system addresses these gaps through role-based access control, faculty-mediated student booking requests, administrative arbitration mechanisms, and constraint-based scheduling validation supported by a relational database backend.

While the current phase emphasizes structured requirements engineering and system modeling, future implementation and deployment will follow an Agile development lifecycle to enable incremental feature delivery and stakeholder-driven refinement. This work demonstrates how systematic analysis and iterative specification development can establish a strong foundation for scalable academic resource management solutions.

***Index Terms*—room allocation, lecture hall management, requirements engineering, UML modeling, academic scheduling, role-based access control, conflict detection, web-based system, Agile lifecycle**

I. Introduction

Academic institutions face ongoing challenges in efficiently managing physical resources, particularly lecture halls, classrooms, and examination venues. Traditional approaches rely heavily on manual processes involving email exchanges, spreadsheet coordination, and informal communication channels. These methods are prone to errors, conflicts, and lack of transparency, often resulting in double bookings, suboptimal resource utilization, and administrative burden.

A. Motivation

At the Indian Institute of Technology Jodhpur, the Lecture Hall Complex (LHC) booking process exemplifies these challenges. Faculty members, students, and various departments must navigate a complex, ad-hoc system to reserve rooms for classes, events, examinations, and other activities. The absence of a unified platform leads to:

- **Administrative Overhead**: Manual verification and approval of each booking request making the process prone to human errors
- **Limited Visibility**: No real-time view of room availability across the campus
- **Inequitable Access**: No direct process for students to initiate legitimate room booking
- **Poor Accountability**: Difficulty tracking resource usage and resolving disputes
- **Scheduling Complexity**: Manual coordination of timetables, slot systems, and examination seating

B. Objectives

This project aims to develop a comprehensive web-based management system that addresses these challenges through:

1) Centralized room inventory and real-time availability tracking
2) Automated timetable-based classroom pre-allocation

3) Multi-tier booking request workflow with role-based permissions
4) Intelligent conflict detection and resolution mechanisms
5) Support for complex slot systems and student enrollment constraints
6) Examination timetable generation and seating arrangement automation (Planned for future implementation)
7) Transparent notification and communication channels

The system is designed to serve multiple user classes—administrators, staff, faculty members, and students—each with tailored interfaces and appropriate access levels. Through seven iterations of requirements refinement, we have developed a mature SRS document that balances functionality, usability, and institutional constraints.

C. Paper Organization

The remainder of this paper is organized as follows: Section II reviews existing solutions and their limitations. Section III details our solution trajectory, including the iterative development of requirements, UML models, and technical architecture. Section IV discusses our analysis process, design decisions, and lessons learned during requirements evolution. Section V presents our conclusions and future work directions.

II. Literature Review and Existing Solutions

A. Current State of Practice

Room booking and resource allocation in academic institutions typically follow one of several approaches:

1) Manual Email-Based Systems: Many institutions, including IIT Jodhpur prior to this project, rely on email exchanges between requesters and administrative staff. While simple to implement, this approach suffers from:

- No centralized availability information
- High latency in request processing
- Prone to communication errors and lost requests
- Difficult to audit or generate usage reports
- No automated conflict detection

2) Spreadsheet-Based Coordination: Some institutions maintain shared spreadsheets (e.g., Google Sheets, Excel) where users can view and request bookings. This offers slight improvements in visibility but introduces new problems:

- Concurrent editing conflicts
- No enforcement of business rules
- Manual conflict checking required
- Limited access control capabilities
- Difficult to integrate with other systems (timetables, enrollment data)

3) Commercial Resource Management Systems: Several commercial platforms exist for facility and room booking:

EMS (Event Management System): A comprehensive solution offering room booking, event scheduling, and resource management. However, it is:

- Expensive for academic institutions
- Designed primarily for corporate environments
- Lacks academic-specific features (slot systems, enrollment-based conflict detection)
- Requires significant customization

CollegeNET R25/25Live: Specifically designed for academic scheduling, offering robust features for course scheduling and room allocation. Limitations include:

- High licensing costs
- Complex configuration and training requirements
- Limited flexibility for institution-specific workflows
- Often restricted to administrative staff access

Ad Astra: Another academic scheduling platform with optimization algorithms. Drawbacks include:

- Primarily focused on course scheduling, not general room booking
- May not support student-initiated requests
- Integration challenges with existing systems

B. Open-Source Solutions

1) Booked Scheduler: An open-source web-based booking system offering:

- **Pros**: Free, customizable, supports recurring bookings, email notifications
- **Cons**: Generic design not tailored to academic constraints, no timetable integration, limited support for complex approval workflows

2) OpenACS Calendar: Part of the OpenACS platform:

- **Pros**: Open-source, extensible architecture
- **Cons**: Dated interface, steep learning curve, requires significant development effort for academic-specific features

C. Gap Analysis

Our review reveals several critical gaps in existing solutions:

1) **Student Access**: Most systems restrict booking initiation to faculty or staff, limiting student organizations' ability to request rooms for legitimate activities.
2) **Academic-Specific Constraints**: Commercial systems lack built-in support for:
   - Multiple slot systems (different for first-year vs. senior students)
   - Enrollment-based conflict detection
   - Faculty-verified student requests
3) **Conflict Resolution Workflows**: Existing solutions offer limited support for handling competing booking requests for the same resource.
4) **Timetable Integration**: Most systems treat room booking as independent from course timetables, requiring duplicate data entry.
5) **Cost and Customization**: Commercial solutions are expensive and difficult to customize, while open-source options require extensive development.

These gaps motivated our decision to develop a custom solution tailored to our institution's specific needs, incorporating lessons learned from both commercial and open-source alternatives.

III. Solution Trajectory

A. Requirements Engineering Process

Our development followed an iterative requirements engineering approach, resulting in seven major revisions of the SRS document over a two-week period (January 19 to February 4, 2026). Each iteration addressed specific stakeholder feedback, technical constraints, or design insights.

1) Version History Analysis: Table I summarizes the evolution of our requirements across versions.

TABLE I
SRS Version Evolution

Ver	Date	Key Changes
1.0	Jan 19	Initial SRS document
1.1	Jan 20	Added REQ-4.1.4 to REQ-4.1.9 (booking validation)
1.2	Jan 21	Added Section 4.3 (slot/venue change)
1.3	Jan 28	Added requirement priorities, formal NFR template, traceability matrix
2.0	Jan 30	**Major revision**: includes student-initiated booking with faculty verification (REQ-4.1.5)
2.1	Feb 4	Updated RTM to reflect v2.0 changes
2.2	Feb 4	Revised conflict handling: admin-driven resolution (REQ-4.3.14)
2.3	Feb 4	Enhanced UML diagrams with detailed attributes and user interactions
2.4	Feb 14	Modified the following UML diagrams: Class, activity, use case. Created the following new diagrams: timing, component.

2) Critical Design Decisions: **Decision 1: Student Booking Model (v2.0)**

Initial designs restricted room booking to faculty or particular student representatives. After stakeholder consultation, we revised this to allow student-initiated requests with mandatory faculty verification (Now mentioned in REQ-4.1.5). This decision addressed:

- **Inclusivity**: Enables student organizations to formally request rooms
- **Accountability**: Faculty verification ensures legitimacy
- **Workload Distribution**: Reduces direct burden on LHC staff for low-risk bookings

The workflow now requires:

1) Student submits booking request with purpose and faculty verifier
2) Selected faculty member reviews and approves/rejects
3) Approved requests forward to LHC staff for final decision

Decision 2: Conflict Resolution Strategy (v2.2)

Initially, we considered peer-to-peer negotiation for conflicting requests. However, this proved impractical due to:

- Power imbalances between requesters
- Potential for protracted negotiations
- Lack of authoritative resolution mechanism

We revised to admin-driven arbitration (Now mentioned in REQ-4.3.14):

- System detects conflicts and escalates to Admin
- Admin reviews both requests and selects one
- System notifies all parties of the decision
- Provides clear audit trail

Decision 3: Graceful Degradation (v2.2, NFR-5.4.1)

Recognizing that student enrollment data might not always be available, we introduced graceful degradation:

- System attempts student-level conflict validation when data is available
- If unavailable, bypasses this check and clearly informs user
- Other validations (room availability, capacity) still enforced
- Prevents system blocking due to missing optional data

B. Functional Requirements Summary

Our final specification includes 28 functional requirements organized into five major features:

1) **Room Booking and Allocation** (REQ-4.1.1 to REQ-4.1.10): Core booking functionality with multi-tier approval, real-time availability, and conflict detection
2) **Timetable-Based Preallocation** (REQ-4.2.1 to REQ-4.2.2): Automated classroom assignment from uploaded timetables
3) **Slot and Venue Change** (REQ-4.3.1 to REQ-4.3.14): Faculty-initiated modifications with constraint validation
4) **Examination Scheduling** (REQ-4.4.1 to REQ-4.4.3): Automated exam timetable and seating generation
5) **Notifications** (REQ-4.5.1 to REQ-4.5.3): Real-time status updates and reminders

Each requirement is labeled with priority (High/Mid/Low) based on:

- Critical path dependencies
- User impact
- Implementation complexity
- Regulatory or policy mandates

C. Non-Functional Requirements

We defined 9 non-functional requirements across four categories:

- **Performance** (NFR-5.1.1): 95% of availability queries within 2 seconds
- **Safety** (NFR-5.2.1): Multi-step confirmation for data deletion
- **Security** (NFR-5.3.1, NFR-5.3.2): Role-based access control, encrypted data transmission and storage
- **Robustness** (NFR-5.4.1 to NFR-5.4.4): Graceful handling of missing data, no crashes, clear user notifications

Each NFR follows a standardized template:

- Description of quality attribute
- Measurable criterion
- Method of verification
- Priority level
- Affected modules

D. UML Modeling

Comprehensive UML models were developed to represent the structural and behavioral aspects of the Unified Room Allocation and Lecture Hall Complex Management System.

1) Use Case Diagram: The Use Case Diagram shows interactions between Student, Faculty, Staff, and Central Admin. It includes booking-related use cases such as Request Event Booking, Request Room Booking, and Approve or Reject Booking Request.

Administrative functions include managing rooms, users, slots, holidays, and timetables. Mandatory processes are modeled using `<<include>>` relationships, while optional alternatives are represented using `<<extend>>` relationships.

2) Class Diagram: The diagram illustrates an object-oriented system architecture comprising approximately 18 classes, interconnected through inheritance and association relationships. The structure is organized into three primary logical groupings:

- **User Hierarchy**: A distinct inheritance structure on the left, featuring a base User class extended by specialized roles (likely Admin, Staff, Faculty, and Student).
- **Core Management Entities**: The central cluster representing the primary domain objects such as Room, Course, and Booking, which form the backbone of the scheduling system.
- **Auxiliary & Support Classes**: Peripheral classes handling specific operational details, including Notifications, Event Types, and Temporal definitions (AcademicYear, Holidays).

3) Activity Diagrams: Two activity diagrams represent the booking lifecycle.

The Booking Request Submission flow covers request initiation, validation, availability checking, optional alternative suggestions, and routing to the appropriate approver based on requester type.

The Booking Approval Workflow models faculty and staff review stages, status updates (approved/rejected), room reservation upon approval, and notification to the requester.

4) Timing Diagram: The Timing Diagram presents the time-based state transitions of Requester, BookingService, RoomService, Staff/Faculty, and NotificationService during booking processing. It shows progression from request submission and validation to approval, room reservation, status update, and final notification, highlighting synchronized state changes across components.

5) Sequence Diagram: The Sequence Diagram models the interaction flow for room/event booking with faculty validation. It covers request submission, validation, availability checking, conditional approval routing (Faculty and Staff), room reservation upon approval, and notification delivery, including alternative and rejection paths.

6) Component Diagram: The Component Diagram shows the modular system architecture. BookingService coordinates RoomService, UserService, TimetableService, and NotificationService through defined interfaces. All components interact with the Database for persistent storage.

E. Requirements Traceability Matrix (RTM)

The RTM (Appendix D of the SRS) maps each requirement to:

- Source/system goal
- Related use case(s)
- Planned software module
- Verification artifact (test case ID or evaluation method)

This ensures bidirectional traceability from high-level goals through requirements to design modules and test cases.

F. Proposed Technical Architecture

1) Technology Stack: **Frontend**:

- Tailwind CSS, JavaScript (ES6+)
- React.js for dynamic UI components
- Bootstrap or Material UI for responsive design
- Chart.js for visualization (availability heatmaps, usage statistics)

Backend:

- Node.js with Express.js
- RESTful API design for modularity
- JWT-based authentication
- Role-based access control middleware

Database:

- PostgreSQL for relational data storage
- Redis for session management and caching availability queries (target: sub-second response)

Deployment:

- Docker containers for consistent development and production environments
- Nginx as reverse proxy

2) System Architecture: The system follows a three-tier architecture:

1) **Presentation Tier**: Web-based GUI with role-specific dashboards
2) **Application Tier**: Business logic layer handling booking workflow orchestration, conflict detection, timetable parsing, and notification dispatching.
3) **Data Tier**: Relational database with normalized schema for users, room inventory, bookings, timetables, and audit logs.

3) Key Algorithms Tested, Results and Other Discussions: To evaluate the efficiency and robustness of the scheduling component of the system, we implemented and experimentally tested three alternative approaches for handling room availability validation and booking conflict detection: a Naive Solver, a Sorted Scheduler, and a BitMask-based Scheduler. These approaches were evaluated primarily on execution time, scalability with increasing booking load, and suitability for integration with a database-backed deployment. There results are compared in Fig 1.

The Naive Solver maintained booking intervals as unsorted lists and performed exhaustive linear overlap checks whenever availability queries or booking requests were issued. While

simple to implement and useful for early prototyping, empirical tests showed that execution time increased significantly as the number of bookings grew. This behaviour matched the expected theoretical complexity of linear interval scanning. The approach was therefore deemed unsuitable for realistic deployment scenarios involving frequent queries or moderate concurrency.

```
# Naive interval scheduler example

bookings = []

def book(interval):
for s, e in bookings:
if interval[0] < e and s < interval[1]:
return False
bookings.append(interval)
return True
```

The Sorted Scheduler improved upon this by maintaining booking intervals in sorted order, allowing faster conflict checks using ordered traversal and partial search. Experimental results indicated noticeable improvements in average response time compared to the naive method, particularly when booking density increased. However, insertion operations incurred additional overhead due to maintaining sorted structures, and performance gains were moderate rather than transformative.

```
# Sorted interval scheduler example

import bisect

bookings = []

def book(interval):
i = bisect.bisect_left(bookings, interval)
if i>0 and bookings[i-1][1] > interval[0]:
return False
if i < len(bookings) and
bookings[i][0] < interval[1]:
return False
bookings.insert(i, interval)
return True
```

The BitMask Scheduler discretized the fixed weekday booking window into equal time slots and represented room occupancy using binary arrays. This allowed constant-time checks for slot availability and extremely fast booking validation. Experimental results consistently showed superior performance for both availability queries and booking operations, especially under high booking loads. This matched theoretical expectations since slot-based indexing eliminates interval search overhead. Additionally, this representation aligned naturally with the operational constraints of the system, where bookings typically occur in fixed-duration academic slots.

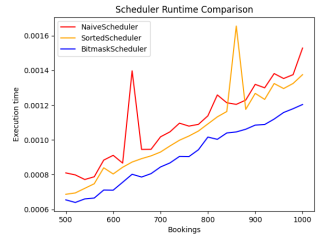

Fig. 1. Plot comparing the runtime of these three algorithms

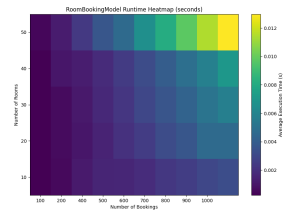

Fig. 2. BitMask runtime against of rooms and of bookings

```
# Bitmask slot scheduler example

SLOTS = 20
schedule = [0]*SLOTS

def book(start,end):
if any(schedule[i]==1 for i in range(start,end)):
return False
for i in range(start,end):
schedule[i] = 1
return True
```

Based on this we decided to perform further tests on the bitmask version. We modified the algorithm a little to also suggest alternative rooms in that time period (as required in the second part of REQ-4.1.9). This was done by searching the availability of all rooms in that time. This is a naive linear search but will work if not most of the rooms are booked. We see in the Fig 2. that bitmask works well with increasing booking densities upto a particular limit before showing some slowness. We are currently thinking of going with Bitmasks given the actual scale of our project.

However for scale-proofness and reliability in case of high booking density we are thinking of also testing alternatives such as *Interval Trees, Segment Trees,* and *Precomputed Availability Index*. We will continue testing these algorithms in our current sprint.

Also, we were thinking of implementing the following model. Define an interface $roomData$ with the following methods:

- `find()` $\rightarrow$ true/false: searches for an element in the container.
- `insert()`: inserts an element into the container.
- `delete()`: deletes an element from the container.

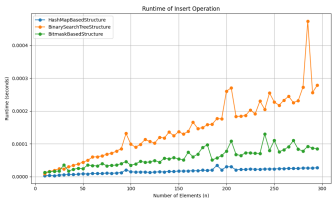

Fig. 3. Runtime for insertion

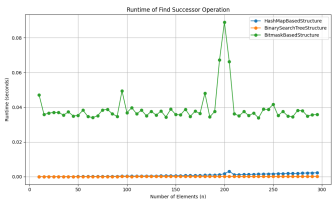

Fig. 4. Runtime for findNext

- `findNext()` → returns the smallest element greater than the input in the container.

For this interface, we thought of the following Data Structures:

- HashMaps
- BitMasks
- Binary Trees

Their performances have been compared in the plot shown in Fig 3, 4, 5 6.

These experimental performances are in line with the theoretical predictions. HashMaps and Bitmasks perform better for insert, delete and search tasks O(1), while Binary tree takes over in findNext task o(log(n)). We also researched about van Emde Boas Trees. These find the perfect tradeoff between Hashmaps and Binary trees in terms of theoretical complexity (O(log(log(Universe Space))). However these are quiet complicated recursive which might be a little overkill for our project, and finally thought of deciding against it. We were thinking of instantiating a $roomData$ object per classroom venue. Free slots will be inserted into the object; booked slots will be removed; search operation will check if the room is free during a slot; findNext will find the next slot during which this classroom is free (in accordance with REQ-4.1.9). This was an alternate method we were thinking about (i.e maintain a container per classroom).

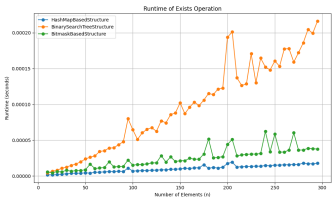

Fig. 5. Runtime for Search(X)/Exists(x)

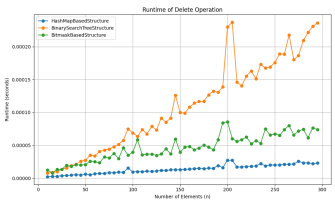

Fig. 6. Runtime for Delete

We will be deciding between the various algorithms and Data Structures in our current sprint.

Links for codes used for generating these plots has been given in Appendix D.

Timetable Import:

- Parses Excel file with course-slot-room-instructor data
- Validates against room inventory and slot definitions
- Batch inserts classroom allocations
- Reports any data inconsistencies for admin review

G. Development Methodology

1) Agile Scrum Framework: We adopted Scrum plan: Scrum team members are written in brackets against each task.

Sprint 1 (Requirements & Design) - 3 week (Completed):

- Stakeholder interviews [Rewant, Aryan, Suparn]
- SRS iterations 1.0 to 2.3 [Rewant, Aryan]
- UML modeling [Suparn, Anshika]
- RTM development [Rewant, Anshika]

Sprint 2 (Requirements Finalization & Technical Preparation) - 2 week (Ongoing):

- Finalizing Software Requirements Specification (SRS) [Rewant]
- User authentication and role management [Suparn, Aryan]
- Testing of algorithms to be implemented [Rewant, Anshika]

Sprint 3 (Core Functionality Implementation) - 2 week:

- Room inventory and availability tracking [Suparn, Rewant, Aryan]
- Basic booking request workflow [Aryan, Anshika]
- Timetable upload and parsing module [Suparn, Aryan, Rewant]
- Conflict detection and alternative suggestion [Suparn, Rewant, Anshika]
- Faculty verification for student requests [Aryan, Anshika]

Sprint 4 (Advanced Features) - 2 week:

- Slot and venue change requests [Rewant, Aryan]
- Examination scheduling [Suparn, Aryan, Anshika]
- Advanced notifications [Rewant, Anshika]

Future Sprints will be decided based on progression of currently defined sprints.

2) Alpha Card Progression: Team members were responsible for specific Alpha aspects, with the respective Alpha managers indicated in brackets and the corresponding progress summarized below:

- **Requirements** (Aryan, Suparn, Rewant, Anshika): Stakeholders identified; user classes defined; requirements prioritized (High/Mid/Low); NFRs measurable; RTM completed; use cases and UML documented; version history maintained.
- **Software System** (Aryan): System scope defined; integration points specified; 3-tier architecture finalized; technology stack selected; performance and security constraints addressed.
- **Work Opportunity** (Suparn): Problem statement defined; objectives listed; gap analysis completed; sprint plan prepared; task allocation performed.
- **Stakeholders** (Rewant): Stakeholder groups recognized; decision authority confirmed; feedback incorporated; engagement tracked.
- **Way of Working** (Rewant): Agile Scrum adopted; sprint structure defined; Git version control used; coding standards and process guidelines documented.
- **Team** (Anshika): Team members identified; roles assigned; Alpha ownership distributed; collaboration structure established.

H. Coding and Quality Standards

- **Version Control**: Git with feature branching and mandatory pull request reviews.
- **Code Style**: ESLint and Prettier for JavaScript (ES6+) to ensure consistent React and Node.js code structure.
- **Backend Structure**: Layered Express.js architecture (routes, controllers, middleware) following RESTful design principles.
- **Testing**: Jest for unit testing and Supertest for API validation, targeting at least 80% code coverage.
- **API Documentation**: Swagger (OpenAPI) for standardized REST API documentation.
- **Security**: JWT-based authentication, role-based access control (RBAC), and input validation for secure request handling.
- **CI/CD**: Automated linting, testing, and Docker builds using GitHub Actions.

IV. Analysis and Discussion

A. Requirements Evolution Insights

Our iterative process revealed several important lessons about requirements engineering in practice.

1) Stakeholder Alignment: The most significant requirement change—introducing student-initiated bookings with faculty verification (v2.0)—emerged from recognizing a mismatch between system capabilities and institutional culture. Initial designs mirrored the existing process, restricting students. However, discussions with student representatives revealed:

- Student organizations frequently need rooms for legitimate academic and cultural activities
- Faculty advisors are well-positioned to verify these requests
- Overly restrictive systems would perpetuate existing inequities

This highlights the importance of inclusive stakeholder engagement beyond just current administrators.

2) Constraint Discovery: The graceful degradation requirement (NFR-5.4.1, v2.2) arose from a late realization that student enrollment data integration might be delayed or unavailable. Rather than treating this as a blocker, we designed the system to operate in a degraded mode—still providing value through room availability and capacity checks while clearly communicating the limitation to users.

This demonstrates the importance of identifying and documenting assumptions about external dependencies, designing for partial failure rather than binary success/failure, and explicit communication of system limitations to users.

3) Priority Refinement: Version 1.3 introduced formal priority levels (High/Mid/Low) for all requirements. This exercise forced us to:

- Distinguish must-have features from nice-to-have enhancements
- Identify dependencies (e.g., conflict detection requires availability tracking)
- Plan phased releases (Phase 1: core booking, Phase 2: slot changes and possibly exam scheduling in future)

High-priority requirements cluster around core booking functionality, conflict prevention, and security—the minimum viable product for replacing the manual system. Mid-priority requirements enhance usability (alternative suggestions, event type templates). Low-priority requirements support advanced workflows (exam scheduling, metadata-based recommendations).

B. UML Modeling Trade-offs : Detail vs. Clarity

Early UML diagrams (v1.x) were abstract and omitted many attributes/methods. While easier to read, they provided insufficient guidance for implementation. Version 2.x added detailed attributes, data types, and method signatures. This improved utility for developers but risked clutter.

We balanced this through:

- Separating high-level overview diagrams from detailed specifications
- Using consistent naming conventions and visibility modifiers

C. Technical Architecture Decisions

1) Database Design Considerations: We chose a relational database (PostgreSQL) over NoSQL alternatives for several reasons:

- **Data Structure**: Room bookings, timetables, and user relationships are inherently relational with strong referential integrity requirements
- **Query Patterns**: Heavy use of joins (e.g., finding available rooms requires joining Rooms, Bookings, Slots, and potentially Courses/Enrollments)

- **ACID Properties**: Critical for preventing double bookings and ensuring consistent state during concurrent requests
- **Maturity**: Well-established tools for backup, replication, and performance tuning

Redis caching addresses the 2-second response time requirement for availability queries by storing frequently accessed availability matrices.

2) Performance Optimization Strategies: To meet NFR-5.1.1 (95% of availability queries under 2 seconds):

- **Database Indexing**: Composite indexes on (room_id, date, slot_id) for fast availability lookups
- **Caching Layer**: Redis cache stores availability grids, invalidated on booking updates
- **Query Optimization**: Use database views for complex joins, avoid N+1 query antipattern
- **Pagination**: Limit result sets, implement lazy loading for UI
- **Asynchronous Processing**: Decouple notification dispatch from critical path

Load testing during development will validate these strategies under realistic concurrent user scenarios.

3) Security Architecture: NFR-5.3.1 and NFR-5.3.2 mandate strong security:

- **Authentication**: JWT tokens with short expiration, refresh token mechanism
- **Authorization**: Middleware validates user role against endpoint requirements
- **Hashing**: TLS 1.3 for data in transit, AES-256 for sensitive data at rest (if needed)
- **Input Validation**: Parameterized SQL queries to prevent injection, sanitize user inputs
- **Audit Logging**: Immutable logs of all booking actions with user ID, timestamp, and outcome

Regular security audits and penetration testing will verify these measures.

D. Challenges and Lessons Learned

1) Requirements Volatility: Seven major revisions in 16 days suggests high requirements volatility. Contributing factors:

- Diverse stakeholder base with competing interests
- Evolving understanding of technical constraints
- Discovery of edge cases through modeling exercises

This reinforces the value of iterative requirements engineering and maintaining traceability to manage change impact.

2) Scope Creep Prevention: Several desirable features (metadata-based recommendations, detailed usage analytics, mobile app) were deferred to future phases to avoid scope creep. We maintained a separate "Future Enhancements" backlog to capture these ideas without committing to immediate implementation.

3) Cross-Functional Communication: Effective collaboration required:

- Regular meetups to synchronize progress
- Shared documentation in version control (GitHub)
- Use of UML and RTM as communication artifacts
- Alpha card assignments to distribute responsibility while maintaining team cohesion

E. Validation Strategy

We planned a multi-faceted validation approach:

Requirements Validation:

- Stakeholder review sessions to confirm SRS accuracy
- Prototyping key workflows (booking, approval) for early feedback

Design Validation:

- Peer review of UML diagrams for correctness and completeness

Implementation Validation:

- Test cases derived from RTM verification artifacts
- Load testing to validate performance NFRs
- Security audit to validate security NFRs
- Usability testing with representative users from each role

F. Comparison with Existing Solutions

Table II compares our proposed system against existing alternatives.

TABLE II
FEATURE COMPARISON WITH EXISTING SOLUTIONS

Feature	**Manual**	**EMS**	**R25**	**Our System**
Real-time Availability	No	Yes	Yes	Yes
Student-Initiated Booking	No	No	Limited	Yes (verified)
Timetable Integration	No	Limited	Yes	Yes
Conflict Detection	Manual	Yes	Yes	Yes (multi-level)
Slot System Support	N/A	No	Limited	Yes (multiple)
Faculty Verification	N/A	N/A	N/A	Yes
Cost	Free	High	High	Free (OSS)
Customizability	N/A	Limited	Limited	High
Enrollment-Based Validation	No	No	Basic	Yes (optional)
Admin Conflict Resolution	Informal	Basic	Basic	Structured

Our system uniquely combines academic-specific features (slot systems, enrollment validation) with inclusive access (student requests) and structured workflows (faculty verification, admin arbitration).

V. Conclusion and Future Work

A. Summary of Contributions

This paper has presented the design and requirements engineering process for a Unified Room Allocation and Lecture Hall Complex Management System. Our key contributions include:

1) A comprehensive SRS document evolved through seven iterations, demonstrating systematic requirements refinement
2) A novel student booking workflow combining accessibility with accountability through faculty verification
3) Graceful degradation architecture enabling operation despite missing external data sources
4) Detailed UML models (use case, class, sequence, activity, timing) providing implementation-ready specifications
5) A complete Requirements Traceability Matrix linking goals to requirements to design modules to test cases
6) Comparative analysis revealing gaps in existing solutions and justifying our design decisions

B. Current Status

As of February 15, 2026, we have completed:

- Requirements engineering and SRS documentation (v2.3)
- High-level system architecture and technology stack selection (tentative)
- UML modeling capturing structure and behavior
- Sprint planning for development phases

C. Next Steps

1) Immediate Focus:

- Finalization and freezing of the Software Requirements Specification (SRS)
- Implementation of user authentication and role-based access control
- Validation and testing of core algorithms prior to integration

2) Core System Implementation:

- Development of room inventory and real-time availability tracking
- Implementation of the booking request workflow
- Timetable upload and automated parsing functionality
- Conflict detection with alternative room/slot suggestion
- Faculty verification mechanism for student booking requests

3) System Enhancement and Extension:

- Slot and venue change request functionality
- Examination scheduling module
- Advanced notification mechanisms
- System testing, refinement, and performance stabilization

D. Lessons for Future Projects

Our experience offers several takeaways for similar projects:

1) **Inclusive Stakeholder Engagement**: Consult end users (students, faculty) early and often, not just administrators
2) **Iterative Requirements Engineering**: Expect significant evolution; use version control and traceability to manage change
3) **Design for Degradation**: Identify external dependencies and design fallback behaviors
4) **Formalize NFRs**: Use structured templates with measurable criteria and verification methods
5) **Comprehensive Modeling**: Invest in detailed UML diagrams—they clarify requirements and guide implementation
6) **Prioritization Discipline**: Distinguish must-have from nice-to-have to control scope

E. Broader Impact

Beyond IIT Jodhpur, this system can be adapted for other academic institutions facing similar resource management challenges. The open-source nature of our planned implementation will enable customization for institution-specific constraints. We envision a community-driven project where universities contribute features, bug fixes, and best practices.

F. Concluding Remarks

The Unified Room Allocation and LHC Management System demonstrates how systematic software engineering practices—requirements engineering, UML modeling, traceability, and architectural design—can address real-world operational challenges. By combining technical rigor with stakeholder collaboration, we have designed a system that balances functionality, usability, security, and institutional constraints. As we proceed to implementation, the solid foundation established through requirements engineering will guide development and testing, increasing the likelihood of delivering a system that truly meets user needs.

Acknowledgment

The authors would like to thank the administration of IIT Jodhpur, particularly the LHC staff and faculty advisors, for their valuable input during requirements elicitation. We also acknowledge the feedback from student organization representatives that shaped the inclusive booking model.

Portions of this manuscript were refined using AI-based language tools for grammar and clarity improvement. All technical decisions, system design, modeling artifacts, experiments, and implementation planning were carried out by the authors.

References

[1] K. E. Wiegers and J. Beatty, *Software Requirements*, 3rd ed. Redmond, WA: Microsoft Press, 2013.
[2] IEEE Computer Society, "IEEE Recommended Practice for Software Requirements Specifications," IEEE Std 830-1998, 1998.
[3] I. Sommerville, *Software Engineering*, 10th ed. Boston, MA: Pearson, 2015.

[4] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 8th ed. New York, NY: McGraw-Hill Education, 2014.
[5] I. Jacobson, G. Booch, and J. Rumbaugh, *The Unified Software Development Process*. Reading, MA: Addison-Wesley, 1999.
[6] G. Booch, J. Rumbaugh, and I. Jacobson, *The Unified Modeling Language User Guide*, 2nd ed. Boston, MA: Addison-Wesley Professional, 2005.
[7] EMS Software, "EMS Event and Space Management Platform," 2023. [Online]. Available: https://www.emssoftware.com/
[8] CollegeNET, "Series25 Academic Scheduling and Event Management," 2023. [Online]. Available: https://www.collegenet.com/series25/
[9] Ad Astra Information Systems, "Platinum Analytics and Academic Scheduling," 2023. [Online]. Available: https://www.aais.com/
[10] Booked Scheduler, "Open Source Resource Scheduling," 2023. [Online]. Available: https://www.bookedscheduler.com/
[11] G. J. Nutt, "The evolution toward flexible workflow systems," *Distributed Systems Engineering*, vol. 3, no. 4, pp. 276–294, 1996.
[12] N. Russell, W. M. P. van der Aalst, and A. H. M. ter Hofstede, "Workflow patterns: The definitive guide," MIT Press, 2005.
[13] J. Desel and W. Reisig, "Place/transition Petri nets," in *Lectures on Petri Nets I: Basic Models*, W. Reisig and G. Rozenberg, Eds. Berlin: Springer, 1998, pp. 122–173.
[14] W. M. P. van der Aalst, A. H. M. ter Hofstede, B. Kiepuszewski, and A. P. Barros, "Workflow patterns," *Distributed and Parallel Databases*, vol. 14, no. 1, pp. 5–51, 2003.
[15] A. Cockburn, *Writing Effective Use Cases*. Boston, MA: Addison-Wesley, 2001.

article

Appendix A
UML Artifacts

Latest UML diagrams are available at:

- https://docs.google.com/document/d/1UrJ5XnMLRnA7pjGx5XzrTDmzIGO5XYlhiyvLbKu4u78/edit?usp=sharing

The complete UML artifacts, including use case, class, activity, sequence, and timing diagrams along with their revision history, are available at:

- https://docs.google.com/spreadsheets/d/1PCXo94bqj9IQ7lp-paRw1r1-4bMyM3jV-pQ5QE47hXw/edit?usp=sharing

Appendix B
Alpha Cards

The Alpha card checklist, progress tracking, and ownership details are maintained at:

- https://docs.google.com/spreadsheets/d/1j-jCB5i2kutlaGNBqRwUtbNTELHUes5O0QoHVLnHzLo/edit?usp=sharing

Appendix C
Requirements Traceability Matrix (RTM)

The complete Requirements Traceability Matrix mapping system goals, requirements, UML artifacts, modules, and verification methods is available at:

- https://docs.google.com/spreadsheets/d/1l4zxEMFL1sghVKOab8W2L6eIrBhoipZTW_bZk0W_9Z4/edit?usp=sharing

Appendix D
Software Requirements Specification

Latest Software Requirements Specification is available at:

- https://docs.google.com/document/d/1fUZ57auyaeIk5vRAjBvNF9S7BO18QaFFbvgHs9sSxtc/edit?usp=sharing

Appendix E
Code used for generating graphs

The code used for generation of graphs for comparing different algorithms is available at:

- https://colab.research.google.com/drive/1O9ejKmBnvwGBdpRyiEYF3FBzJ2Td0GZk#scrollTo=8p6hV45gUqrV
- https://colab.research.google.com/drive/1pgr74TJsBxywbanAwMo_V4fA7x8De1s8#scrollTo=0ec4c3a7